

UNIVERSIDAD LA SALLE



Facultad de Ingenierías y Arquitectura
Escuela Profesional de Ingeniería de Software

Curso: Calidad de Software
Docente: Maribel Molina Barriga

PRÁCTICA DE LABORATORIO 02

Evaluación del diseño mediante atributos de calidad y evidencias

Proyecto OLA — Observatorio Litoral de
Anomalías térmicas

*Informe técnico con diagnóstico, rediseño, matriz de hallazgos y
registro de decisión de arquitectura*

Integrantes

Frederick Mares Graos
Jhordan Huamani Huamani
Jorge Ortiz Castañeda
Miguel Angel Flores Leon
Piero Adrian Delgado Chipana

Arequipa, octubre de 2026

Índice

1. Datos de la práctica y alcance	2
1.1. Adaptación de la guía al proyecto	2
1.2. Control previo: herramientas y entorno	3
2. Diagnóstico	3
2.1. Modelo del diseño actual	3
2.2. Lista de revisión	4
2.3. Métricas de diseño	7
2.4. Fragmentos que evidencian los problemas	8
3. Hallazgos y priorización	9
4. Mejora: rediseño y refactorización	9
4.1. R1: canales de aviso	13
4.2. R2: almacén, despachador y reloj	13
4.3. Métricas después del cambio	14
4.4. Pruebas	16
5. Comparación mediante un escenario de cambio	16
6. Documentación	18
6.1. Registro de decisión de arquitectura	18
6.2. Registro de evidencias	19
6.3. Limitaciones de la evaluación	19
7. Verificación de cierre	20
8. Ejercicios propuestos	20
8.1. Aplicar el diseño a un nuevo escenario de cambio	20
8.2. Proponer una prueba que evidencie comprobabilidad	21
8.3. Comparar dos alternativas de refactorización con una matriz ponderada	21
9. Conclusiones	22
Referencias	22

1. Datos de la práctica y alcance

Caso evaluado	Proyecto autorizado: OLA, aplicación web que clasifica la anomalía de la temperatura del mar de los laboratorios costeros de IMARPE y avisa a los usuarios suscritos.
Módulo evaluado	«Alertas y avisos» del backend (requisitos RF-01 y RF-03). Ocupa el lugar del módulo Matrícula que propone la guía.
Versión evaluada	Repositorio Miguel777-rgb/Proyecto_Calidad_de_Software, rama main, commit d5c054f del 13/09/2026.
Fecha de ejecución	Revisión del diseño: 05/10/2026. Mediciones en el entorno del proyecto: 06/10/2026.
Unidades revisadas	14 unidades de diseño (sección 2.2) en 12 módulos de Python. Las métricas cubren además <code>api.deps</code> : 13 módulos.
Fuera de alcance	El frontend (React), los módulos de importación, series y proyección, y las pruebas de extremo a extremo.

1.1 Adaptación de la guía al proyecto

El backend de OLA está escrito en Python con FastAPI y organiza buena parte de su lógica en **módulos de funciones**, no en clases. Para aplicar la guía se tomó como unidad de diseño la clase cuando existe (`SmtplibMailer`, `Notification`, `Clock`) y el módulo cuando agrupa funciones (`notification_service`, `alerts_repo`). En los diagramas UML los módulos se representan como clases de utilidad con el estereotipo «module», y las métricas «por clase» de la guía se calcularon por módulo: número de funciones y métodos, dependencias y complejidad.

El cuestionario de la guía se responde en un documento aparte.

1.2 Control previo: herramientas y entorno

Herramienta	Versión	Uso en la práctica
Sistema operativo	CachyOS (<i>rolling</i>), núcleo Linux 7.2.8	Equipo anfitrión de los contenedores.
Docker	29.8.1	Contenedores de la API, de la base de pruebas y de $\text{\LaTeX}$.
Git	2.55.0	Obtener el código, aplicar los parches y medir con <code>git diff --numstat</code> .
Imagen ola-api (Python)	3.13.15	Imagen de desarrollo del proyecto (backend/Dockerfile); ejecuta las pruebas, ruff y mypy.
pytest · pytest-cov · coverage	9.1.1 · 7.1.0 · 7.16.0	Pruebas unitarias y de integración; cobertura de sentencias.
ruff	0.16.6	Análisis estático y complejidad ciclomática (regla C901).
mypy	2.3.1	Tipos en modo estricto, con el complemento de pydantic.
PostgreSQL	17.11	Base de pruebas ola_test en un contenedor desechable (postgres:17-alpine).
SQLAlchemy · FastAPI · pydantic · psycpg · bcrypt	2.0.52 · 0.141.1 · 2.13.5 · 3.3.5 · 5.0.0	Dependencias del backend fijadas en uv.lock.
Guion de métricas	propio (metricas.py)	LOC, NOM, Ce, Ca, I y complejidad con el módulo ast.
Diagramas UML	Archify 3.0.1 (sistema visual) · libsvg 2.62.3	Generador propio con la paleta, la tipografía y las flechas de Archify; exportación a PDF y PNG.
Markdown	—	Redactar el ADR; puede abrirse en StackEdit.
$\text{\LaTeX}$	TeX Live 2026 (pdfTeX 1.40.29)	Componer este informe.

Configuración que puede afectar los resultados. Las pruebas se ejecutaron en la imagen ola-api con las variables `OLA_*` de `.env.example`, salvo `OLA_JWT_SECRET` (64 caracteres hexadecimal aleatorios) y `OLA_ADMIN_PASSWORD`, que se definieron con valores propios, como pide la plantilla. La carpeta `dataset/` se montó en `/dataset`, igual que en `compose.yml`. La base de pruebas corrió en un contenedor PostgreSQL 17 aislado, no en el servicio `db` del proyecto, para no tocar los datos de desarrollo. Con los secretos de la plantilla, la prueba `test_is_production_solo_es_verdadero_en_produccion` falla porque el validador de producción los rechaza; no es un defecto del módulo evaluado.

2. Diagnóstico

2.1 Modelo del diseño actual

Las Figuras 1a y 1b (**E-01**) modelan el módulo. El backend sigue cuatro capas: los routers de la API reciben la petición, los servicios coordinan el caso de uso, el dominio contiene la lógica pura (rachas, textos de los avisos) y los repositorios acceden a PostgreSQL mediante SQLAlchemy.

El flujo evaluado tiene dos pasos. Al evaluar las alertas, `alert_service.evaluate` detecta los episodios y `notification_service.create_for_events` registra un aviso pendiente por cada suscrito y canal. Después, un administrador dispara `notification_service.send_pending`, que envía los correos pendientes y reintenta los fallidos.

Cada caja del diagrama muestra todos los atributos y operaciones de la unidad, de modo que el número de operaciones coincide con su NOM, y lleva sus valores de NOM, Ce y Ca. Las unidades y dependencias con hallazgos están en rosa, con su identificador. Para que el diagrama se lea, no se dibujan las dependencias hacia `api.deps` ni hacia módulos fuera del alcance; Ce las cuenta igualmente. La Figura 1b detalla las 15 clases de `db.models`.

2.2 Lista de revisión

Se aplicó una lista de ocho criterios a 14 unidades (**E-02**, Tabla 1). Los criterios son: C1, responsabilidad única; C2, cohesión interna; C3, depende de abstracciones y no de detalles; C4, abierto a extensión sin modificación; C5, respeta la regla de capas; C6, sin duplicación; C7, complejidad ciclomática de cada función no mayor que 10; C8, comprobable sin infraestructura (base de datos o servidor de correo). «Parc.» indica un cumplimiento parcial.

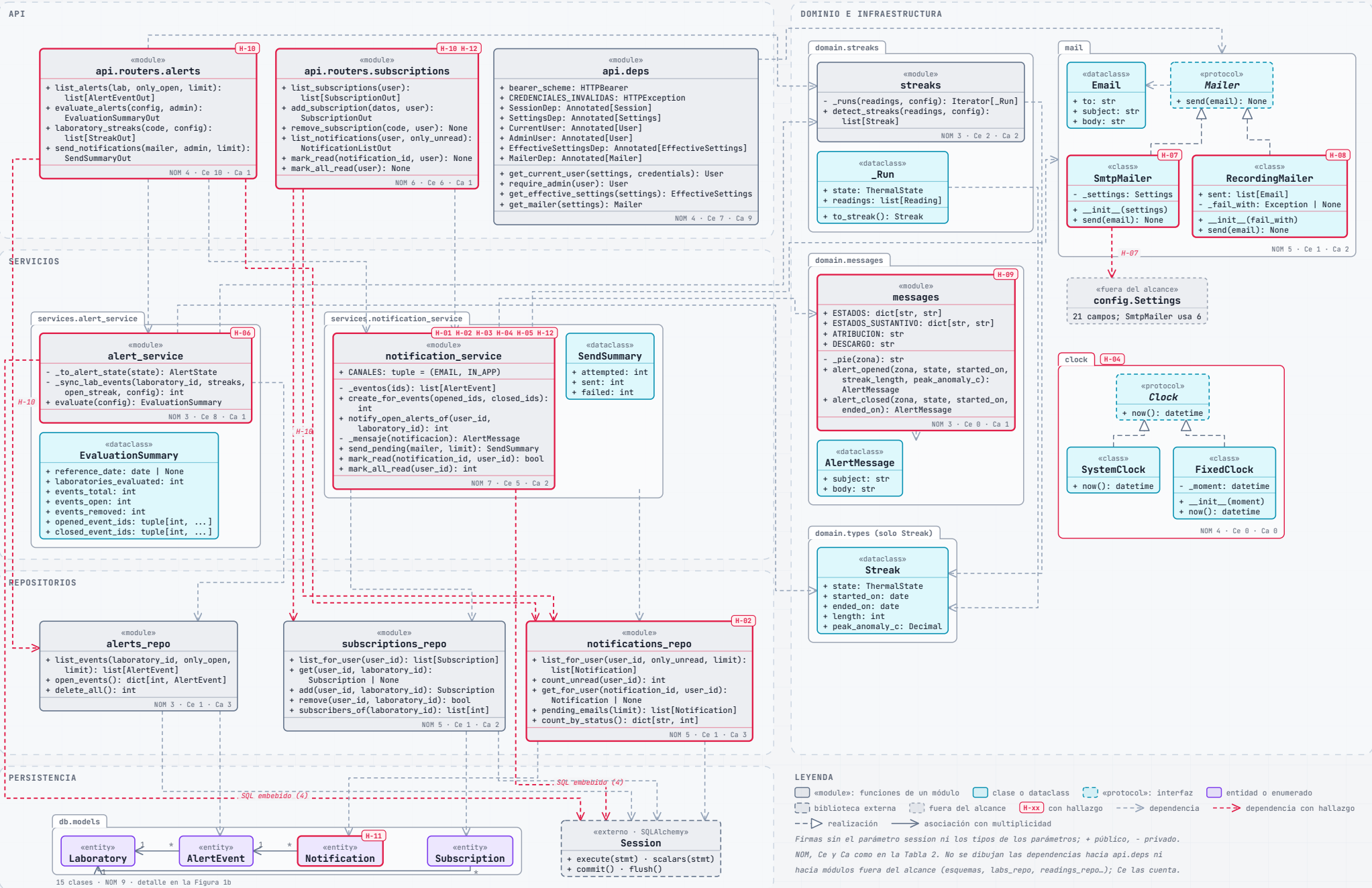
Tabla 1: E-02. Lista de revisión aplicada a 14 unidades

ID	Unidad	C1	C2	C3	C4	C5	C6	C7	C8
U01	<code>services.notification_service</code>	No	Parc.	Parc.	No	No	No	Sí	No
U02	<code>services.alert_service</code>	Parc.	Sí	No	Parc.	No	Sí	Sí	No
U03	<code>repositories.notifications_repo</code>	Sí	Sí	Sí	No	Sí	Parc.	Sí	Parc.
U04	<code>repositories.alerts_repo</code>	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Parc.
U05	<code>repositories.subscriptions_repo</code>	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Parc.
U06	<code>mail.SmtpMailer</code>	Sí	Sí	Parc.	Sí	Sí	Sí	Sí	Parc.
U07	<code>mail.RecordingMailer</code>	Sí	Sí	Sí	Sí	Parc.	Sí	Sí	Sí
U08	<code>mail.Mailer · Email</code>	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Sí
U09	<code>domain.messages · AlertMessage</code>	Sí	Sí	Sí	Parc.	Sí	Parc.	Sí	Sí
U10	<code>domain.streaks · _Run · Streak</code> (<code>domain.types</code>)	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Sí
U11	<code>db.models.Notification</code>	Sí	Sí	N/A	Parc.	Parc.	Sí	Sí	N/A
U12	<code>clock.Clock · SystemClock · FixedClock</code>	Sí	Sí	Sí	Sí	Sí	Sí	Sí	Sí
U13	<code>api.routers.alerts</code>	Parc.	Parc.	Sí	Sí	Parc.	Parc.	Sí	Parc.
U14	<code>api.routers.subscriptions</code>	Parc.	Sí	Sí	Sí	Parc.	Parc.	Sí	Parc.
Incumplimientos (No)		1	0	1	2	2	1	0	2

Resultado: 9 incumplimientos y 24 cumplimientos parciales sobre 112 comprobaciones. Dos unidades concentran 8 de los 9 incumplimientos: `notification_service` (5) y `alert_service` (3). El dominio (U09, U10, U12) y los repositorios (U04, U05) no presentan incumplimientos. Ninguna función supera el umbral de complejidad (C7). La justificación de cada celda está en la hoja «Lista de revisión» de la matriz de hallazgos (`Matriz_Hallazgos_OLA.xlsx`).

Diseño actual del módulo «Alertas y avisos» – diagrama de clases

Commit d5c054f · E-01a · En rosa, las unidades y dependencias con hallazgos (H-xx). Los atributos de las entidades están en la Figura 1b.



LEYENDA

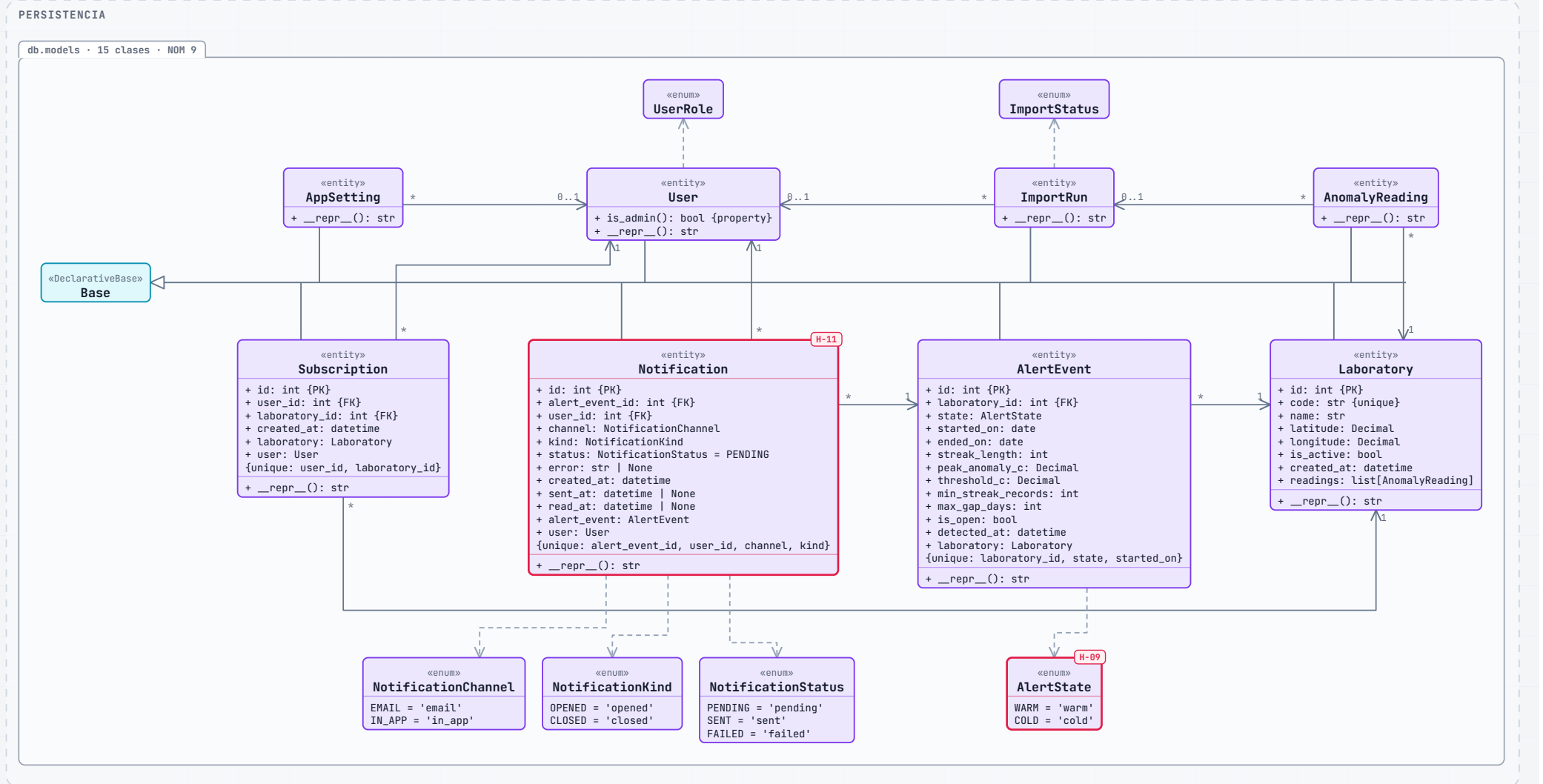
- «module»: funciones de un módulo
- clase o dataclass
- «protocol»: interfaz
- entidad o enumerado
- biblioteca externa
- fuera del alcance
- H-xx con hallazgo
- dependencia
- dependencia con hallazgo
- realización
- asociación con multiplicidad

Firmas sin el parámetro session ni los tipos de los parámetros; + público, - privado.
NOM, Ce y Ca como en la Tabla 2. No se dibujan las dependencias hacia api.deps ni hacia módulos fuera del alcance (esquemas, labs_repo, readings_repo...); Ce las cuenta.

Figura 1a: E-01. Diagrama de clases del diseño actual del módulo «Alertas y avisos». En rosa, las unidades y dependencias con hallazgos.

Diseño actual – las 15 clases de db.models

Commit d5c054f · E-01b · Atributos completos en las clases del módulo «Alertas y avisos»; en las demás, nombre, estereotipo y métodos. NOM 9.



LEYENDA

«module»: funciones de un módulo «class» clase o dataclass «protocol»: interfaz «entity» entidad o enumerado «library» biblioteca externa «out of scope» fuera del alcance «H-xx» con hallazgo «-->» dependencia «-->» dependencia con hallazgo

«-->» realización «-->» generalización «-->» asociación con multiplicidad

Todas las entidades heredan de Base (árbol de generalización). {PK}, {FK} y {unique} vienen de las columnas y restricciones de SQLAlchemy.

Figura 1b: E-01. Las 15 clases de db.models en el diseño actual, con las asociaciones que derivan de sus claves foráneas.

2.3 Métricas de diseño

La Tabla 2 (E-03) recoge las métricas por unidad, obtenidas con el guion propio `metricas.py`, basado en el módulo `ast`, y contrastadas con `ruff`. NOM es el número de funciones y métodos; Ce, los módulos internos de los que depende la unidad (acoplamiento eferente); Ca, los módulos internos que dependen de ella (acoplamiento aferente); $I = Ce / (Ca + Ce)$, la inestabilidad; CCmáx y CCsum, la complejidad ciclomática máxima y acumulada de sus funciones.

Tabla 2: E-03. Métricas de diseño del módulo, versión d5c054f

Unidad (módulo)	LOC	NOM	Ce	Ca	I	CCmáx	CCsum
<code>services.notification_service</code>	141	7	5	2	0.71	5	20
<code>services.alert_service</code>	128	3	8	1	0.89	4	9
<code>repositories.notifications_repo</code>	65	5	1	3	0.25	2	6
<code>repositories.alerts_repo</code>	25	3	1	3	0.25	3	5
<code>repositories.subscriptions_repo</code>	39	5	1	2	0.33	2	7
<code>mail</code>	36	5	1	2	0.33	3	8
<code>domain.messages</code>	57	3	0	1	0.00	1	3
<code>domain.streaks</code>	48	3	2	2	0.50	8	10
<code>db.models</code>	229	9	0	20	0.00	1	9
<code>clock</code>	13	4	0	0	0.00	1	4
<code>api.routers.alerts</code>	92	4	10	1	0.91	3	7
<code>api.routers.subscriptions</code>	81	6	6	1	0.86	3	10
<code>api.deps</code>	57	4	7	9	0.44	4	8
Total	1011	61					106

Lectura de las métricas:

- **Complejidad.** Es baja en todo el módulo: la función más compleja es `streaks._runs`, con 8, y ninguna supera 10. El problema del módulo no es la complejidad algorítmica, sino el reparto de responsabilidades.
- **Cohesión.** `notification_service` es la unidad con mayor complejidad acumulada (20) y la segunda en tamaño tras los modelos. Sus 7 funciones atienden cinco asuntos distintos: decidir qué avisos corresponden, insertarlos, redactar el mensaje, enviarlo y marcar la lectura.
- **Acoplamiento.** Los servicios dependen de 5 y 8 módulos internos y, además, importan SQLAlchemy para construir sentencias: 4 en `notification_service` y 4 en `alert_service`. `routers.alerts` es la unidad más inestable ($I = 0.91$, $Ce = 10$).
- **Abstracción sin uso.** `clock` tiene $Ca = 0$ y $Ce = 0$: ningún módulo de producción lo importa. El reloj que debía hacer deterministas las pruebas no se usa, y `notification_service` llama a `datetime.now(UTC)` en 3 lugares.
- **Núcleo estable.** `db.models` tiene $Ca = 20$ e $I = 0$: casi todo depende de él. Es aceptable porque es declarativo ($CCmáx = 1$), pero obliga a que los enumerados del dominio vivan en la capa de persistencia.

2.4 Fragmentos que evidencian los problemas

El Fragmento 1 (**E-05**) muestra el envío actual. La función obtiene los pendientes con una consulta que solo conoce el correo (línea 2), elige el texto con un condicional (línea 6), envía (líneas 8–10), captura cualquier excepción y sella la hora con el reloj del sistema (línea 18). Para probar el reintento o el registro del fallo hace falta una base PostgreSQL con episodios, usuarios y suscripciones.

Fragmento 1: E-05. Envío en el diseño actual (notification_service.py, líneas 153–179, sin la cadena de documentación).

```
1 def send_pending(session: Session, mailer: Mailer, *, limit: int = 200) -> SendSummary:
2     pendientes = notifications_repo.pending_emails(session, limit=limit)
3     enviados = fallidos = 0
4
5     for notificacion in pendientes:
6         mensaje = _mensaje(notificacion)
7         try:
8             mailer.send(
9                 Email(to=notificacion.user.email, subject=mensaje.subject, body=mensaje.body)
10            )
11        except Exception as exc:
12            notificacion.status = NotificationStatus.FAILED
13            notificacion.error = str(exc)[:500]
14            fallidos += 1
15        else:
16            notificacion.status = NotificationStatus.SENT
17            notificacion.error = None
18            notificacion.sent_at = datetime.now(UTC)
19            enviados += 1
20
21    session.commit()
22    return SendSummary(attempted=len(pendientes), sent=enviados, failed=fallidos)
```

El Fragmento 2 muestra el bloque de inserción que el servicio construye por su cuenta. Aparece dos veces en el mismo archivo, en create_for_events y en notify_open_alerts_of, junto con el diccionario de la fila.

Fragmento 2: E-05. SQL construido en el servicio y duplicado (`notification_service.py`, líneas 114–131; el mismo bloque está en las líneas 71–90).

```
1     filas = [  
2         {  
3             "alert_event_id": evento.id,  
4             "user_id": user_id,  
5             "channel": canal,  
6             "kind": NotificationKind.OPENED,  
7             "status": NotificationStatus.PENDING,  
8         }  
9         for evento in vigentes  
10        for canal in CANALES  
11    ]  
12    creados = session.execute(  
13        insert(Notification)  
14        .values(filas)  
15        .on_conflict_do_nothing(constraint="uq_notification_unica")  
16        .returning(Notification.id)  
17    ).scalars()  
18    return len(list(creados))
```

3. Hallazgos y priorización

La revisión produjo 12 hallazgos (**E-04**). La Tabla 3, en la página siguiente, los presenta ordenados por prioridad con su severidad, su evidencia por archivo y línea, la decisión y el estado. La misma matriz, con las fórmulas de priorización, está en la hoja «Hallazgos» de `Matriz_Hallazgos_OLA.xlsx`.

Regla de priorización. Cada hallazgo se puntuó de 1 a 5 en impacto (cuánto dificulta cambiar o probar el módulo) y en esfuerzo de corrección. Se ordenó primero por impacto y, a igual impacto, por menor esfuerzo. La severidad resume el impacto: alta para 4 y 5, media para 3 y baja para 1 y 2. Los cinco hallazgos priorizados son H-01, H-02, H-05, H-03 y H-04. Los cinco afectan a la misma unidad y comparten causa, de modo que se resuelven con dos refactorizaciones:

- **R1, canales de aviso:** resuelve H-02.
- **R2, almacén, despachador y reloj:** resuelve H-01, H-03, H-04 y H-05.

Los otros siete quedan registrados con su decisión: cinco pendientes (H-06, H-07, H-09, H-10 y H-12) y dos aceptados (H-08, por ahora, y H-11).

4. Mejora: rediseño y refactorización

Las dos refactorizaciones se implementaron sobre una copia del repositorio y están completas en `refactorizacion_avisos.patch`, que se aplica sin conflictos sobre el commit `d5c054f`. Las Figuras 2a y 2b (**E-06**) muestran el diseño resultante: en verde los elementos nuevos, en ámbar los módulos modificados y en rosa los hallazgos que siguen pendientes.

Tabla 3: E-04. Matriz de hallazgos, ordenada por prioridad

ID	Hallazgo	Principio / atributo	Sev.	Imp.	Esf.	Prior.	Evidencia	Decisión	Estado
H-01	notification_service reúne cinco responsabilidades (planificar avisos, persistirlos, redactar el mensaje, enviarlo y marcar la lectura) y construye 4 sentencias SQL sin pasar por el repositorio.	SRP · respeto de capas · modificabilidad	Alta	5	3	1	E-01, E-03, E-05. notification_service.py: 46–55, 85–90, 104–110, 125–130. LOC 141, CCsum 20.	Refactorizar: el servicio solo coordina; la persistencia pasa al repositorio y el envío a un despachador (R2).	Resuelto en el parche
H-02	Los canales no son extensibles: CANALES es una tupla fija, send_pending solo sabe enviar correo y el repositorio filtra por EMAIL en dos consultas. Un canal nuevo obliga a modificar el servicio, el repositorio, el router y las dependencias.	OCP · modificabilidad	Alta	5	3	1	E-01, E-05, E-10. notification_service.py: 36, 153–179; notifications_repo.py: 55–71, 74–80. En el escenario webhook cambian 6 módulos existentes.	Rediseñar con un contrato Channel y un registro de canales (R1). count_by_status, que solo alimenta el resumen del envío, sigue contando el correo.	Resuelto en el parche
H-05	La lógica de envío (reintentos, registro del fallo, límite de la tanda) no puede probarse sin PostgreSQL: tiene 0 pruebas unitarias y 34 de integración.	Comprobabilidad	Alta	4	3	3	E-09. tests/integration/test_notifications.py: 34 pruebas, 12.5–12.9 s en 5 ejecuciones.	Extraer la lógica a NotificationDispatcher, que depende de los protocolos NotificationStore, Channel y Clock (R2).	Resuelto en el parche
H-03	El bloque insert ... on_conflict_do_nothing ... returning y el diccionario de la fila están duplicados en create_for_events y notify_open_alerts_of.	DRY · mantenibilidad	Media	3	1	4	E-05. notification_service.py: 71–90 y 114–131.	Unificar en notifications_repo. insert_pending y domain.notices.plan_rows (R2).	Resuelto en el parche
H-04	La abstracción Clock (con SystemClock y FixedClock) no la importa ningún módulo de producción (Ca = 0). notification_service llama a datetime.now(UTC) en 3 lugares.	DIP · comprobabilidad	Media	3	1	4	E-01, E-03. clock.py: 15–31; notification_service.py: 175, 187, 194.	Injectar Clock en el despachador y en la marca de lectura (R2).	Resuelto en el parche
H-10	El router coordina dos servicios (evaluar y registrar avisos), accede a 4 repositorios directamente y repite 4 veces la búsqueda de laboratorio con su error 404.	SRP · DRY · respeto de capas	Media	3	2	6	E-03. routers/alerts.py: 26–34, 47–58, 78–83; subscriptions.py: 39–44, 62–67. Ce = 10, I = 0.91.	Crear una dependencia LaboratoryDep y un caso de uso evaluate_and_notify.	Pendiente
H-06	alert_service construye 4 sentencias SQL en lugar de usar alerts_repo: la consulta y el borrado de los episodios obsoletos, el upsert de los vigentes y la consulta de los identificadores existentes.	Respeto de capas · mantenibilidad	Media	3	3	7	E-01, E-03. alert_service.py: 60–105 y 163.	Mover el upsert y las consultas a alerts_repo.	Pendiente
H-07	Smtplib recibe el objeto Settings completo (21 campos) y usa 6: acoplamiento de sello. Por esa importación, todo módulo que use mail carga pydantic y la configuración.	ISP · acoplamiento	Baja	2	1	8	E-03, E-07. mail.py: 14, 31–46; config.py: 23–60.	Pasar una configuración SMTP propia de 6 campos.	Pendiente
H-09	El estado se recibe como str y se traduce con dos diccionarios paralelos; AlertState duplica a ThermalState y exige un conversor.	Obsesión por primitivos · DRY	Baja	2	2	9	messages.py: 16–17, 51–52; alert_service.py: 40–41.	Usar ThermalState en los mensajes y un solo enumerado.	Pendiente
H-12	Los límites de transacción son inconsistentes: create_for_events, send_pending, mark_read y mark_all_read confirman dentro del servicio, pero notify_open_alerts_of no lo hace y confirma el router que lo llama.	Consistencia · mantenibilidad	Baja	2	2	9	notification_service.py: 92, 96–131, 178, 188, 197; routers/subscriptions.py: 50–51.	Fijar la transacción en un solo nivel (router o caso de uso). El parche lo documenta en notify_open_alerts_of, pero no lo corrige.	Pendiente
H-11	El canal se guarda como tipo enumerado de PostgreSQL: añadir un canal exige una migración en cualquier diseño.	Modificabilidad	Baja	2	3	11	E-10. models.py: 309–316; migración 9c022526c786: 44–46.	Aceptar; reevaluar si se añaden canales con frecuencia.	Aceptado
H-08	RecordingMailer, un doble de prueba, vive en un módulo de producción.	Cohesión	Baja	1	1	12	mail.py: 49–59.	Mover a tests/ cuando se toquen las pruebas.	Aceptado por ahora

Severidad: Alta = impacto 4–5; Media = 3; Baja = 1–2. Impacto: 1 = cosmético ... 5 = bloquea cambios previsibles o impide probar. Esfuerzo: 1 = menos de una hora ... 5 = varios días. Prioridad: orden por Impacto × 10 – Esfuerzo (empates en la misma posición). «Resuelto en el parche»: corregido en refactorizacion_avisos.patch y validado (472 pruebas pasan); falta aplicarlo en una rama (ADR-001).

Diseño propuesto del módulo «Alertas y avisos» – diagrama de clases

d5c054f • refactorizacion_avisos.patch • E-06a • Verde: nuevo; ámbar: modificado; rosa: hallazgos pendientes (H-xx). Los atributos de las entidades están en la Figura 2b.

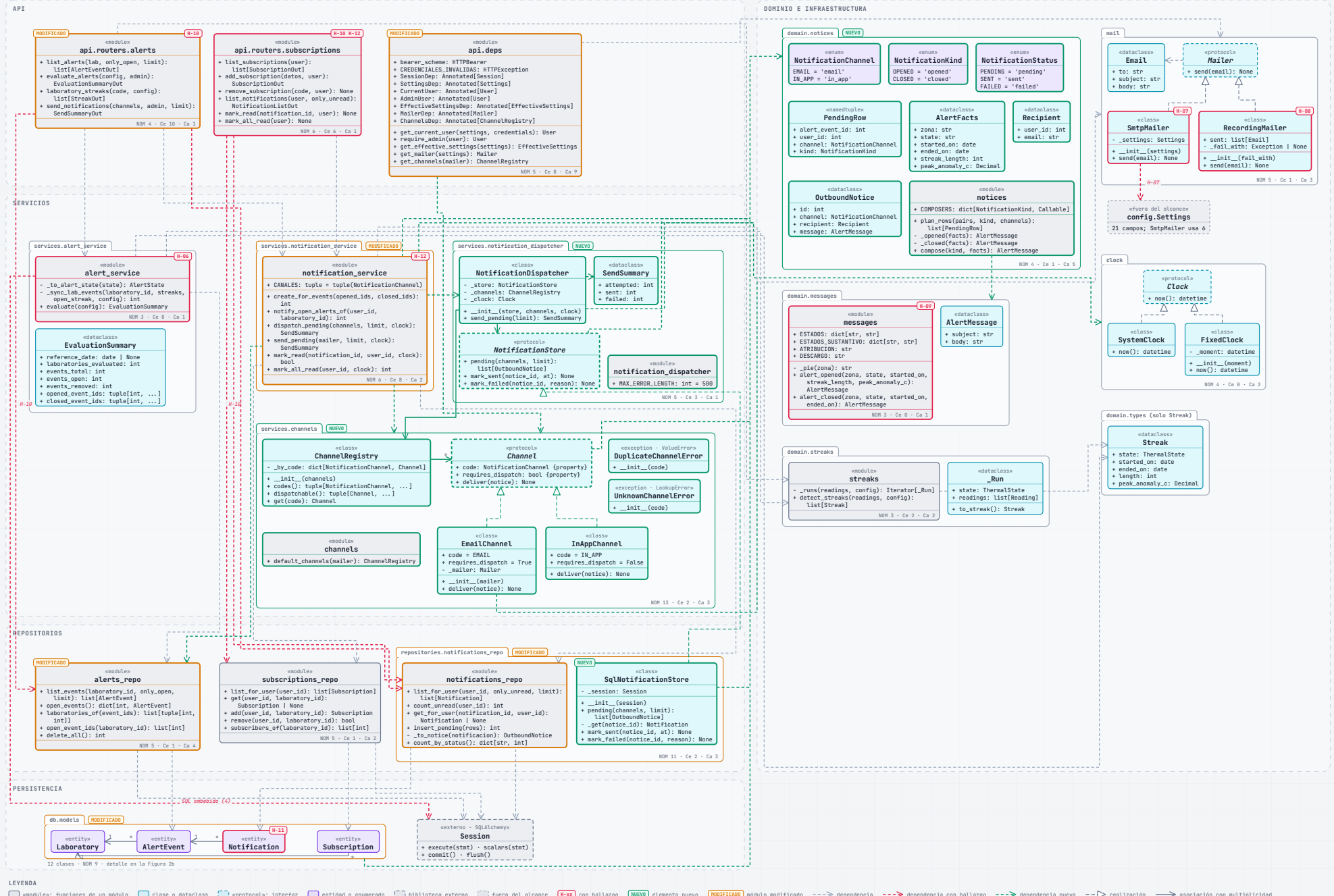


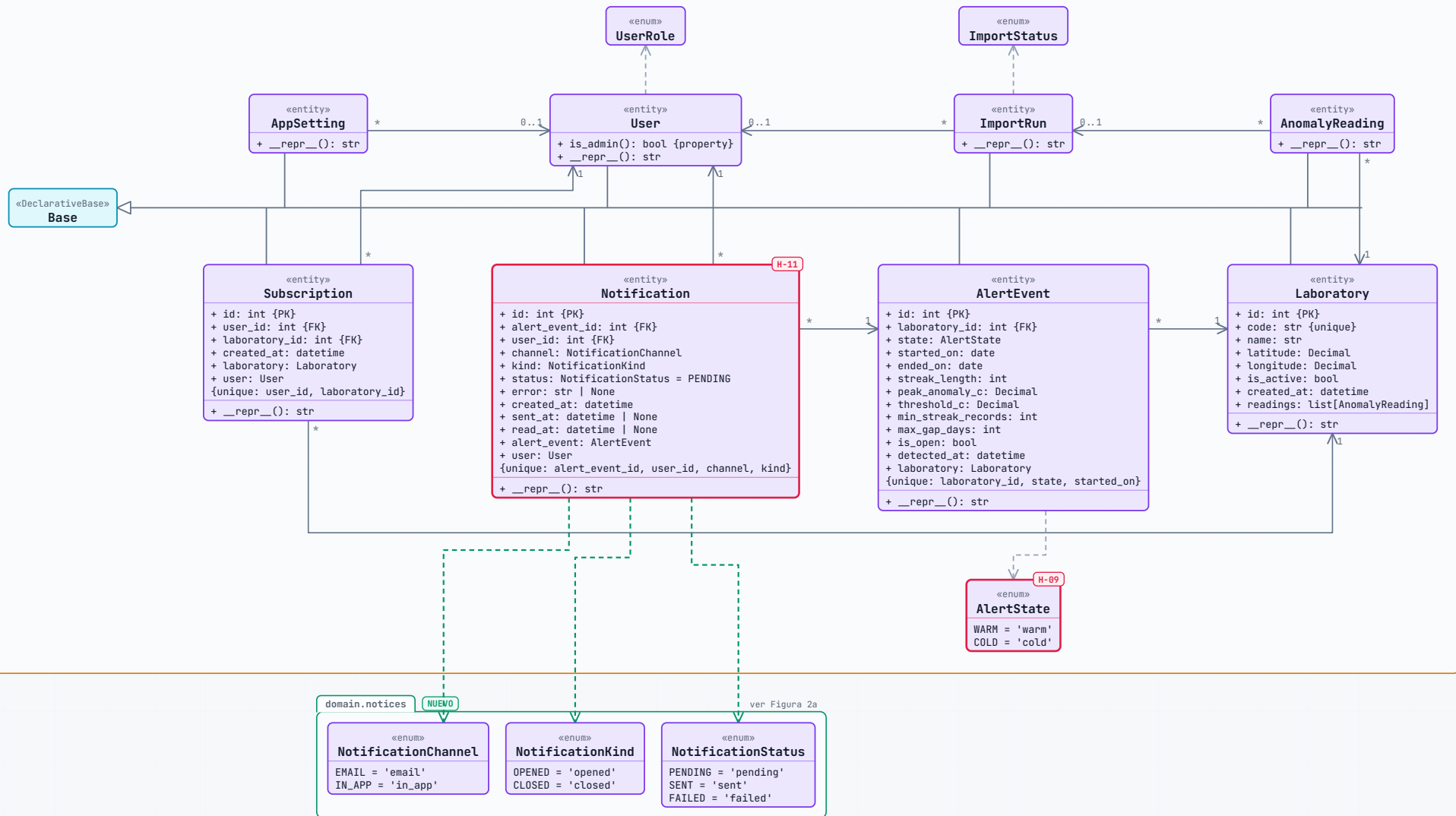
Figura 2a: E-06. Diagrama de clases del diseño propuesto del módulo «Alertas y avisos». Verde: nuevo; ámbar: modificado; rosa: hallazgos pendientes.

Diseño propuesto – las clases de db.models

E-06b · Los enumerados de los avisos pasan a domain.notices y db.models los importa (la persistencia depende del dominio). En rosa, los hallazgos que siguen pendientes.

PERSISTENCIA Y DOMINIO

db.models · 12 clases · NOM 9 **MODIFICADO**



LEYENDA

 «module»: funciones de un módulo
 clase o dataclass
 «protocol»: interfaz
 entidad o enumerado
 biblioteca externa
 fuera del alcance
 H-xx con hallazgo
 MODIFICADO módulo modificado
 NUEVO elemento nuevo

-.-> dependencia
 -.-> dependencia con hallazgo
 -.-> dependencia nueva
 --> realización
 --> generalización
 --> asociación con multiplicidad

Todas las entidades heredan de Base (árbol de generalización). {PK}, {FK} y {unique} vienen de las columnas y restricciones de SQLAlchemy.

4.1 R1: canales de aviso

Problema. Que el sistema tenga dos canales está escrito en tres sitios: la tupla `CANALES`, el cuerpo de `send_pending` y los filtros del repositorio. Incumple el principio abierto-cerrado: añadir un canal obliga a modificar código que ya funciona.

Cambio. Se introduce el contrato `Channel` (Fragmento 3, **E-07**). Cada canal sabe entregar un aviso y declara si requiere envío: el correo sí; el aviso dentro de la aplicación no, porque el registro guardado ya es la entrega. `ChannelRegistry` indexa los canales y `default_channels` es el único punto donde se decide cuáles tiene el sistema.

Fragmento 3: E-07. Contrato de canal y canal de correo (`services/channels.py`).

```

1 class Channel(Protocol):
2     @property
3     def code(self) -> NotificationChannel: ...
4
5     @property
6     def requires_dispatch(self) -> bool:
7         """Falso en los canales de consulta, donde el aviso guardado ya es la entrega."""
8         ...
9
10    def deliver(self, notice: OutboundNotice) -> None:
11        """Entrega el aviso. Lanza una excepcion si falla; quien llama la registra."""
12        ...
13
14
15    class EmailChannel:
16        code = NotificationChannel.EMAIL
17        requires_dispatch = True
18
19        def __init__(self, mailer: Mailer) -> None:
20            self._mailer = mailer
21
22        def deliver(self, notice: OutboundNotice) -> None:
23            self._mailer.send(
24                Email(to=notice.recipient.email, subject=notice.message.subject,
25                    body=notice.message.body)
26            )
27
28
29    def default_channels(mailer: Mailer) -> ChannelRegistry:
30        """Punto unico donde se decide que canales tiene el sistema."""
31        return ChannelRegistry([EmailChannel(mailer), InAppChannel()])

```

El mismo criterio se aplicó al texto del aviso. El condicional de `_mensaje` se sustituye por la tabla `COMPOSERS` de `domain/notices.py`, que asocia cada tipo de aviso con su redactor. Un tipo nuevo se añade en esa tabla.

4.2 R2: almacén, despachador y reloj

Problema. La decisión de qué hacer cuando un envío falla está mezclada con las consultas a la base y con el reloj del sistema, así que no puede probarse de forma aislada.

Cambio. La lógica de envío pasa a `NotificationDispatcher` (Fragmento 4, **E-07**). El despachador recibe por el constructor tres colaboradores definidos como protocolos: el almacén de avisos,

el registro de canales y el reloj. No importa directamente SQLAlchemy, FastAPI ni smtplib. La implementación real del almacén, `SqlNotificationStore`, vive en el repositorio junto con `insert_pending`, que concentra la inserción antes duplicada.

Fragmento 4: E-07. Despachador con dependencias inyectadas (`services/notification_dispatcher.py`).

```

1  class NotificationStore(Protocol):
2      def pending(
3          self, channels: Sequence[NotificationChannel], *, limit: int
4      ) -> list[OutboundNotice]: ...
5
6      def mark_sent(self, notice_id: int, at: datetime) -> None: ...
7
8      def mark_failed(self, notice_id: int, reason: str) -> None: ...
9
10
11 class NotificationDispatcher:
12     def __init__(self, store: NotificationStore, channels: ChannelRegistry, clock: Clock) ->
13         None:
14         self._store = store
15         self._channels = channels
16         self._clock = clock
17
18     def send_pending(self, *, limit: int = 200) -> SendSummary:
19         codigos = [canal.code for canal in self._channels.dispatchable()]
20         pendientes = self._store.pending(codigos, limit=limit)
21         enviados = fallidos = 0
22
23         for aviso in pendientes:
24             try:
25                 self._channels.get(aviso.channel).deliver(aviso)
26             except Exception as exc:
27                 self._store.mark_failed(aviso.id, str(exc)[:MAX_ERROR_LENGTH])
28                 fallidos += 1
29             else:
30                 self._store.mark_sent(aviso.id, self._clock.now())
31                 enviados += 1
32
33         return SendSummary(attempted=len(pendientes), sent=enviados, failed=fallidos)

```

`notification_service` queda como coordinador (Fragmento 5): compone el despachador, fija el límite de la transacción y conserva sus firmas públicas; `send_pending` sigue existiendo y delega en `dispatch_pending`. Por eso las 34 pruebas de integración existentes no necesitaron cambios. El único router que cambia es `routers/alerts.py`, en 3 líneas: el envío recibe los canales por inyección (`ChannelsDep`, definido en `api/deps.py`) en lugar del `Mailer`. Los enumerados de canal, tipo y estado se movieron de `db/models.py` a `domain/notices.py`; la capa de persistencia los importa del dominio, y no al revés.

4.3 Métricas después del cambio

- `notification_service` baja de 141 a 73 líneas y su complejidad acumulada de 20 a 12. Deja de construir sentencias SQL (de 4 a 0).
- `domain.notices` y `clock` se importan sin ninguna biblioteca externa, y el despachador y los canales se cargan sin SQLAlchemy, FastAPI, Starlette ni pycpg; se comprobó

Fragmento 5: E-07. El servicio como coordinador (services/notification_service.py).

```

1 def notify_open_alerts_of(session: Session, user_id: int, laboratory_id: int) -> int:
2     vigentes = alerts_repo.open_event_ids(session, laboratory_id)
3     filas = plan_rows(
4         ((event_id, user_id) for event_id in vigentes), NotificationKind.OPENED, CANALES
5     )
6     return notifications_repo.insert_pending(session, filas)
7
8
9 def dispatch_pending(
10     session: Session, channels: ChannelRegistry, *, limit: int = 200, clock: Clock | None =
11     None
12 ) -> SendSummary:
13     despachador = NotificationDispatcher(
14         SqlNotificationStore(session), channels, clock or SystemClock()
15     )
16     resumen = despachador.send_pending(limit=limit)
17     session.commit()
18     return resumen

```

Tabla 4: E-08. Métricas antes → después de los módulos afectados

Módulo	LOC	NOM	Ce	Ca	CCmáx	CCsum
services.notification_service	141 → 73	7 → 6	5 → 8	2 → 2	5 → 4	20 → 12
services.notification_dispatcher	— → 40	— → 5	— → 3	— → 1	— → 4	— → 8
services.channels	— → 55	— → 13	— → 2	— → 3	— → 3	— → 16
domain.notices	— → 72	— → 4	— → 1	— → 5	— → 1	— → 4
repositories.notifications_repo	65 → 115	5 → 11	1 → 2	3 → 3	2 → 2	6 → 14
repositories.alerts_repo	25 → 41	3 → 5	1 → 1	3 → 4	3 → 3	5 → 8
db.models	229 → 222	9 → 9	0 → 1	20 → 19	1 → 1	9 → 9
clock	13 → 13	4 → 4	0 → 0	0 → 2	1 → 1	4 → 4
mail	36 → 36	5 → 5	1 → 1	2 → 3	3 → 3	8 → 8
api.deps	57 → 61	4 → 5	7 → 8	9 → 9	4 → 4	8 → 9
Total medido	1011 → 1173	61 → 91				106 → 138

importándolos con esas bibliotecas bloqueadas (**E-07**). No son del todo independientes de la infraestructura: channels importa mail, que importa la configuración, así que el despachador y los canales cargan pydantic y smtplib. Es el acoplamiento que describe H-07.

- clock pasa de $Ca = 0$ a $Ca = 2$: la abstracción que existía empieza a usarse.
- El costo es más código. Los módulos medidos pasan de 1 011 líneas en 13 módulos a 1 173 en 16 (+16 %), y notification_service depende ahora de 8 módulos internos en lugar de 5, porque coordina piezas que antes contenía. El acoplamiento no desaparece: cambia de dependencias sobre detalles (sentencias SQL, reloj del sistema) a dependencias sobre contratos.

4.4 Pruebas

Tabla 5: E-09. Pruebas, cobertura y análisis estático, antes y después

Comprobación	Antes (d5c054f)	Después (parche)
Pruebas unitarias	212 pasan	242 pasan (212 previas y 30 nuevas)
Pruebas de integración	230 pasan, 0 omitidas	230 pasan, 0 omitidas
Pruebas unitarias de la lógica de envío	0	12
Tiempo de las pruebas del envío (5 ejecuciones)	12.5–12.9 s (34 de integración)	0.02 s (12 del despachador); 0.04 s (las 30 nuevas)
Cobertura de sentencias del backend	95.11 %	95.39 %
Cobertura de los 16 módulos del alcance	95.63 %	96.32 %
<code>ruff check .</code>	Sin observaciones	Sin observaciones
<code>mypy src</code> (estricto, con el complemento de <code>pydantic</code>)	0 errores en 61 archivos	0 errores en 64 archivos
Parche aplicado sobre d5c054f	—	<code>git apply --check</code> sin conflictos; 472 pasan

Las dos pruebas que dependen del dataset real se ejecutan, porque `dataset/` está montado en `/dataset` como en el contenedor del proyecto. `ruff format --check` señala un único archivo sin formatear, `tests/integration/test_auth_service.py`, ajeno al módulo e igual antes y después. Las 34 pruebas de integración del envío tardan lo mismo antes y después: el cambio no las acelera, sino que añade 12 pruebas que comprueban la misma lógica sin base de datos, en una fracción del tiempo.

5. Comparación mediante un escenario de cambio

Escenario. «Añadir un tercer canal de aviso: una pasarela de mensajería externa a la que el sistema entrega cada aviso mediante una petición HTTP (*webhook*).» Se eligió porque la SRS ya retiró un canal (el SMS) por su costo y un canal distinto del correo es el cambio más previsible del módulo.

Especificación. Para que la comparación sea justa, las dos implementaciones cumplen la misma especificación. Cada episodio registra también un aviso para el canal `WEBHOOK`. El envío que dispara el administrador lo entrega por HTTP POST, con la misma política de fallos y reintentos que el correo. Si `OLA_WEBHOOK_URL` no está configurada, la entrega falla con «pasarela no configurada» y el aviso queda para reintento, como si el servidor de correo estuviera caído.

Método. El mismo cambio se implementó dos veces, sobre el diseño actual y sobre el propuesto, y se midió con `git diff --numstat` y con la suite existente, sin modificarla y con la base de pruebas recreada (E-10). El cliente de la pasarela (`webhook.py`, 49 líneas) y el parámetro de configuración son idénticos en ambos diseños. Los dos cambios se entregan como parches aplicables en `escenario_webhook/`.

Clases afectadas. En el diseño actual cambian `db/models.py` (un miembro del enumerado), `notification_service.py` (tupla de canales, firma y cuerpo de `send_pending`), `notifications_repo.py` (la consulta de pendientes), `api/deps.py` (la dependencia de la pasarela), `routers/alerts.py` (un parámetro nuevo) y `config.py`. En el diseño propuesto

Tabla 6: E-10. El mismo cambio en los dos diseños

Indicador	Diseño actual	Diseño propuesto
Módulos existentes modificados	6	4
Líneas añadidas	88	87
Líneas de código existente eliminadas o reescritas	13	4
Líneas tocadas en el servicio de avisos, que contiene el bucle de envío	28	0
Líneas tocadas en el repositorio de avisos	6	0
Líneas tocadas en el router	6	0
Complejidad ciclomática del bucle de envío (era 4 en ambos)	6	4
Pruebas existentes que dejan de pasar	11 de 442	12 de 472
de ellas, cuentan los avisos registrados por episodio	6	6
de ellas, comprueban el resumen del envío	5	5
de ellas, fijan qué canales requieren envío	0	1

cambian `domain/notices.py` (un miembro del enumerado), `services/channels.py` (la clase `WebhookChannel` y su registro en `default_channels`), `api/deps.py` y `config.py`. El despachador, el servicio, el repositorio y el router quedan intactos. Los Fragmentos 6 y 7 muestran dónde entra el canal en cada diseño.

Fragmento 6: E-10. El canal nuevo en el diseño actual: una rama más en el bucle de envío (`escenario_webhook/disenio_actual.patch`).

```

1  try:
2      if notificacion.channel is NotificationChannel.WEBHOOK:
3          if webhook is None:
4              raise WebhookNotConfiguredError
5          webhook.send(
6              WebhookMessage(
7                  recipient=notificacion.user.email,
8                  subject=mensaje.subject,
9                  body=mensaje.body,
10             )
11         )
12     else:
13         mailer.send(
14             Email(to=notificacion.user.email, subject=mensaje.subject, body=mensaje.body)
15         )

```

Interpretación. La mejora en modificabilidad es real, pero moderada, y se ve en la localidad del cambio: dos módulos existentes menos, nueve líneas reescritas menos y ninguna línea tocada en el servicio, el repositorio ni el router. La diferencia más clara está en la complejidad. En el diseño actual cada canal nuevo añade ramas al bucle de envío, que pasa de 4 a 6; en el propuesto, la lógica del canal vive en `WebhookChannel.deliver` y el despachador no cambia.

En cambio, el rediseño no reduce las pruebas que hay que actualizar. En ambos diseños dejan de pasar las mismas 11, porque cuentan los avisos de cada episodio o el resumen del envío, que cambian por especificación al añadir un canal siempre activo. El diseño propuesto suma una más, `test_solo_el_correo_requiere_envio`, que fija qué canales requieren envío y también debe actualizarse. Ninguna de esas pruebas falla por un defecto: describen un comportamiento que el

Fragmento 7: E-10. El canal nuevo en el diseño propuesto: una clase que cumple el contrato y se registra (escenario_webhook/disenio_propuesto.patch).

```

1 class WebhookChannel:
2     """Entrega el aviso a la pasarela de mensajería externa."""
3
4     code = NotificationChannel.WEBHOOK
5     requires_dispatch = True
6
7     def __init__(self, sender: WebhookSender | None) -> None:
8         self._sender = sender
9
10    def deliver(self, notice: OutboundNotice) -> None:
11        if self._sender is None:
12            raise WebhookNotConfiguredError
13        self._sender.send(
14            WebhookMessage(
15                recipient=notice.recipient.email,
16                subject=notice.message.subject,
17                body=notice.message.body,
18            )
19        )

```

cambio modifica a propósito.

Lo que el rediseño no resuelve. En ambos diseños hace falta una migración de Alembic para el tipo enumerado notification_channel (H-11).

6. Documentación

6.1 Registro de decisión de arquitectura

La decisión principal se documenta en ADR-001-canales-y-despachador-de-avisos.md (E-11). Su contenido, en resumen:

- **Contexto.** notification_service reúne cinco responsabilidades; los canales no son extensibles, el servicio construye SQL, el envío no se prueba sin PostgreSQL y Clock no se usa.
- **Opciones.** A, puertos y canales; B, cambio mínimo (mover el SQL y añadir un condicional por canal); C, cola asíncrona con un trabajador.
- **Decisión.** Opción A, con 4.25 puntos en la matriz ponderada frente a 3.55 y 3.20.
- **Consecuencias positivas.** Un canal nuevo no toca el servicio, el despachador, el repositorio ni el router, y la complejidad del bucle de envío no crece; 30 pruebas unitarias sin base de datos; el servicio baja a la mitad de líneas.
- **Consecuencias negativas.** 16 % más de código, más indirección y un registro de canales que hay que mantener. No reduce las pruebas que hay que actualizar al añadir un canal.
- **Estado.** Aceptado el 06/10/2026 por el Product Owner; queda pendiente aplicar el parche en una rama.

6.2 Registro de evidencias

Las salidas en bruto de las mediciones (pytest, ruff, mypy, cobertura, tiempos y métricas) están en la carpeta evidencias/, nombradas con el identificador de la evidencia que respaldan.

ID	Evidencia	Archivo o ubicación	Cómo se obtuvo
E-01	Diagrama de clases del diseño actual	Figuras 1a y 1b; diagramas/E-01a..., E-01b...	Generador propio con el sistema visual de Archify
E-02	Lista de revisión de 14 unidades	Tabla 1; hoja «Lista de revisión»	Lectura del código
E-03	Métricas de diseño antes	Tabla 2; hoja «Métricas»; E-03_metricas_antes.txt	metricas.py, ruff
E-04	Matriz de hallazgos	Tabla 3; hoja «Hallazgos»	Revisión estructurada
E-05	Fragmentos de código antes	Fragmentos 1 y 2	Commit d5c054f
E-06	Diagrama de clases del diseño propuesto	Figuras 2a y 2b; diagramas/E-06a..., E-06b...	Generador propio con el sistema visual de Archify
E-07	Código después e importaciones sin infraestructura	Fragmentos 3 a 5; refactorizacion_avisos.patch; E-07_importaciones...	Refactorización; importación con bibliotecas bloqueadas
E-08	Métricas de diseño después	Tabla 4; hoja «Métricas»; E-08_metricas_despues.txt	metricas.py, ruff
E-09	Pruebas, cobertura y análisis estático	Tabla 5; hoja «Pruebas y cobertura»; E-09_*	pytest, pytest-cov, ruff, mypy
E-10	Escenario de cambio en ambos diseños	Tabla 6; Fragmentos 6 y 7; escenario_webhook/; E-10_*	git diff --numstat, pytest
E-11	Registro de decisión de arquitectura	Archivo ADR-001 (Markdown)	Redacción en Markdown

6.3 Limitaciones de la evaluación

- **Base de datos fuera de Compose.** PostgreSQL 17.11 corrió en un contenedor desechable con la imagen de la API del proyecto, no en el servicio db de compose.yml; la versión mayor es la misma.
- **Configuración de plantilla.** Se usó .env.example con secretos propios. Con los valores de la plantilla falla una prueba de configuración ajena al módulo.
- **Sin extremo a extremo.** No se ejecutaron las pruebas de Playwright.
- **Métricas por módulo.** Al tratar los módulos como clases no se calculó una métrica formal de cohesión (LCOM); la cohesión se evaluó con la lista de revisión.
- **Escenario prototipo.** El canal por *webhook* no incluye la migración de la base ni una pasarela real; sirve para medir el cambio, no para desplegarse. La especificación elegida (canal siempre activo) determina qué pruebas existentes fallan.
- **Tiempos.** Dependen de la máquina; se informan como el rango de cinco ejecuciones.

- **Diagramas.** Omiten las dependencias hacia `api.deps` y hacia módulos fuera del alcance; Ce las cuenta.
- **Puntuaciones.** El impacto y el esfuerzo de cada hallazgo son estimaciones del equipo.

7. Verificación de cierre

Comprobación	Resultado	Detalle
Los resultados corresponden al alcance y la versión declarados	Cumple	Las mediciones «antes» se tomaron del commit <code>d5c054f</code> , que sigue siendo la versión de <code>main</code> ; el parche y los escenarios se validaron con <code>git apply --check</code> sobre su base.
Cada evidencia es legible, está numerada y se cita en el análisis	Cumple	E-01 a E-11 , citadas en las secciones 2 a 6, listadas en la sección 6.2 y con su salida en bruto en evidencias/.
Las limitaciones y los datos faltantes están registrados	Cumple	Sección 6.3.
Las acciones pendientes están registradas	Cumple	Lista siguiente.

Acciones pendientes

1. Aplicar `refactorizacion_aviso.patch` en una rama del repositorio y fusionarla, según el ADR-001 aceptado.
2. Mover el SQL de `alert_service` a `alerts_repo` (H-06).
3. Fijar el límite de las transacciones en un solo nivel (H-12).
4. Crear una dependencia común para la búsqueda de laboratorio en los routers (H-10).
5. Dar a `Smtplib` una configuración SMTP propia (H-07) y unificar `AlertState` con `ThermalState` (H-09).
6. Hacer que `count_by_status` cuente todos los canales que se envían.

8. Ejercicios propuestos

8.1 Aplicar el diseño a un nuevo escenario de cambio

Escenario. «Avisar cuando una alerta vigente se intensifica», es decir, añadir un tercer tipo de aviso además de inicio y fin.

En el diseño propuesto el cambio se localiza así:

- `domain/notices.py`: un miembro nuevo en `NotificationKind` y una entrada en `COMPOSERS`.
- `domain/messages.py`: la función que redacta el texto.
- `alert_service`: detectar la intensificación, igual en ambos diseños.

- No cambian el despachador, los canales, el repositorio ni el servicio de avisos. La prueba parametrizada `test_todos_los_momentos_tienen_texto` falla si se añade el tipo y se olvida su texto.

En el diseño actual habría que modificar además `notification_service._mensaje`. Esa función trata como aviso de fin todo lo que no sea un aviso de inicio, de modo que, si se olvidara actualizarla, el tipo nuevo enviaría el texto «terminó la alerta» sin que fallara ninguna prueba. El rediseño convierte ese error silencioso en un fallo visible.

8.2 Proponer una prueba que evidencie comprobabilidad

La prueba del Fragmento 8 comprueba una regla de negocio de RF-03: un envío fallido no detiene a los demás. No usa base de datos, servidor de correo ni el reloj del sistema; los tres colaboradores son dobles en memoria. Forma parte de las 30 pruebas nuevas, que pasan en 0.04 s.

Fragmento 8: Prueba aislada con dobles (`tests/unit/test_notification_dispatcher.py`).

```
1 def test_un_fallo_no_detiene_los_demas_envios(self):
2     almacen = AlmacenEnMemoria([aviso(1), aviso(2), aviso(3)])
3     canal = CanalQueFallaCon({2})
4
5     resumen = despachador(almacen, [canal]).send_pending()
6
7     assert (resumen.sent, resumen.failed) == (2, 1)
8     assert canal.entregados == [1, 3]
```

La prueba equivalente del diseño actual (`test_un_fallo_no_detiene_los_demas_envios`, en las pruebas de integración) necesita seis *fixtures*: cliente HTTP, sesión de base de datos, datos de muestra, configuración, cabeceras de usuario y buzón. Además no puede provocar que falle solo el segundo envío, porque el doble de correo falla siempre o nunca.

8.3 Comparar dos alternativas de refactorización con una matriz ponderada

Se compararon tres alternativas: A, puertos y canales (la implementada); B, cambio mínimo, que mueve el SQL al repositorio y añade un condicional por canal; y C, una cola asíncrona con un trabajador en segundo plano. La puntuación va de 1 a 5; en esfuerzo y riesgo, 5 significa menor esfuerzo y menor riesgo.

Tabla 7: Matriz ponderada de alternativas

Criterio	Peso	A. Puertos y canales	B. Cambio mínimo	C. Cola asíncrona
Modificabilidad (canal o tipo de aviso nuevo)	30 %	5	3	5
Comprobabilidad (pruebas sin infraestructura)	25 %	5	2	4
Esfuerzo de implementación	20 %	3	5	1
Riesgo de regresión	15 %	4	5	2
Comprensibilidad para el equipo	10 %	3	4	2
Total ponderado	100 %	4.25	3.55	3.20

La alternativa A obtiene la mayor puntuación porque es la única que mejora a la vez los dos atributos con más peso. B es la más barata y segura, pero deja intactos H-02 y H-05. C iguala a A en modificabilidad, pero su costo y su riesgo no se justifican para un envío que hoy dispara un administrador a mano. Las puntuaciones de B y C son estimaciones: solo A se implementó y se midió. La matriz, con sus fórmulas, está en la hoja «Alternativas» del libro de Excel.

9. Conclusiones

1. El módulo «Alertas y avisos» tiene un dominio bien diseñado (funciones puras, complejidad máxima de 8) y una capa de servicios con problemas concentrados: 8 de los 9 incumplimientos de la lista de revisión están en dos unidades.
2. El problema no era la complejidad, sino la cohesión y la dirección de las dependencias. Ninguna función superaba el umbral de 10, y aun así el servicio de avisos reunía cinco responsabilidades y no podía probarse sin base de datos.
3. Dos refactorizaciones resolvieron los cinco hallazgos priorizados sin modificar ninguna de las 442 pruebas existentes, que siguen pasando en el entorno del proyecto, con mypy estricto sin errores y una cobertura del módulo que sube de 95.63 % a 96.32 %.
4. La mejora se midió con un escenario de cambio implementado en ambos diseños con la misma especificación: el canal nuevo toca dos módulos existentes menos y no aumenta la complejidad del bucle de envío. La misma medición mostró su límite: no reduce las pruebas que hay que actualizar.
5. El rediseño tiene un costo, 16 % más de código y más indirección, y deja pendientes siete hallazgos, registrados con su decisión.

Referencias

- ISO/IEC. (2023). *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
- Nygaard, M. (2011). *Documenting architecture decisions*.
<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- Pressman, R. (2010). *Ingeniería del software: un enfoque práctico* (7.^a ed.). McGraw-Hill.
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson.
- tt-ali. (2026). *Archify* (versión 3.0.1) [Software]. <https://github.com/tt-ali/archify>
- Equipo OLA. (2026). *Repositorio del proyecto OLA*, commit d5c054f.
https://github.com/Miguel777-rgb/Proyecto_Calidad_de_Software